

DALOC

A Cross-Chain Prime Brokerage Platform
for Collateralized Credit

Bartosz Nowak

bartosz@herodotus.dev

[@bartolomeo_diaz](https://twitter.com/bartolomeo_diaz)

Marcello Bardus

marcello@herodotus.dev

[@0xmarcello](https://twitter.com/0xmarcello)

June 2026

Abstract

DALOC is a cross-chain prime brokerage and lending platform. A single collateral deposit provides unified margin, leverage, and risk management across multiple chains and venues. Collateral stays on its home chain and continues accruing native yield; coordination relies on zero-knowledge and storage proofs rather than trusted bridges or relayers.

Three products share one platform. **Borrow** originates fixed-rate, fixed-term loans against collateral on any supported chain. **Prime Trade** opens a credit line from yield-bearing collateral to a perpetuals venue—borrowed funds are restricted to trading by protocol-controlled margin agents. **Earn** gives liquidity providers passive fixed-rate yield: capital is deployed into loans by a vault operator whose strategy is bounded by a proof-enforced mandate.

A unified order book matches borrower intents against solver quotes, settling at the best available rate. No capital is locked before settlement, and criteria-based filtering lets regulated and open participants share one liquidity surface. Each loan executes as an isolated position within a single EVM transaction.

Solvers deploy collateral on established lending protocols rather than a proprietary pool, reserving only hedging capital to convert variable funding to fixed terms—achieving $15\text{--}30\times$ capital leverage and eliminating the cold-start problem from the first loan. Markets are defined by permissionless zero-knowledge programs that govern accrual, liquidation, and collateral policy. Risk is quantified dynamically through simulation, producing per-loan terms rather than static protocol parameters. Loan positions are tokenised as transferable debt instruments with provably accruing interest, giving creditors a liquid, tradeable claim.

Contents

1	Introduction	4
2	Products	4
2.1	Borrow	4
2.2	Prime Trade	5
2.3	Earn	5
3	Protocol Architecture	5
3.1	Order Book	5
3.2	Criteria-Based Matching	5
3.3	Markets	6
3.4	Hook Composition	6
3.5	Solvers	7
4	Settlement and Verification	7
4.1	Epoch Mechanics	7
4.2	Settlement Programs	8
4.3	Market and Loan Contracts	8
4.4	Oracle Freedom and Permissionless Markets	8
4.5	System Invariants	9
5	Loan Lifecycle	9
5.1	Origination	9
5.2	Refinancing	10
5.3	Close and Liquidation	10
6	Capital Management	11
6.1	Vault-Funded Path	11
6.2	Externally-Sourced Path	12
6.3	Capital Efficiency	12
7	Position Tokens and Secondary Markets	13

7.1	Debt Tokens	13
7.2	Secondary Markets	14
8	The Harvest Vault	14
8.1	Operator and Mandate	14
8.2	Compliance Proof	14
8.3	Fee-Gating	15
8.4	NAV and Shares	15
8.5	Epoch Withdrawal	15
8.6	Risk Decomposition	15
9	Risk Quantification	15
9.1	Dynamic Underwriting	16
9.2	Methodology	16
9.3	Pricing	16
10	Cross-Chain Exposure	17
10.1	Deterministic Addressing	17
10.2	Hyperliquid Margin Agent	17
10.3	Effective Leverage	18
11	Governance and Trust Assumptions	18
11.1	Protocol Administration	18
11.2	Liveness Dependencies	18
12	Discussion	19
12.1	Risks	19
12.2	Future Capabilities	19
13	Conclusion	20

1 Introduction

On-chain lending totals nearly \$28 billion in active loans as of mid-2026 [4], yet the dominant pool-based designs [1, 10] serve only variable-rate, open-term use cases. Fixed terms, fixed rates, per-position isolation, and cross-chain collateral lie outside their scope. A holder of yield-bearing assets on one chain who wants leveraged exposure on another must manually compose across protocols, bearing bridging and counterparty risk at each step.

Existing protocols address fragments of the problem. Morpho Blue offers isolated markets on a single chain [6]. Euler v2 adds modular vaults with governor-controlled risk [5]. Midnight introduces fixed-maturity terms but operates per-market, per-chain [11]. Cross-venue prime brokerages aggregate positions under unified margin, gaining netting efficiency but coupling all venue exposure into one solvency computation—a single venue failure can trigger global liquidation.

No existing system simultaneously aggregates liquidity across venues, converts variable funding to fixed terms, isolates per-position risk, and orchestrates cross-chain collateral without trusted bridges. Pool designs also lock capital from the moment of deposit; DALOC’s externally-sourced path commits only hedging capital (3–7% of principal), achieving 15–30× leverage versus 1:1 direct lending. Institutional participants face a further gap: permissionless order books offer no compliance filtering without fragmenting into separate permissioned forks.

DALOC addresses all four requirements on a single platform layer: a unified order book with epoch-based ZK settlement, a Harvest Vault funding loans directly or via external venues, atomic hook chains for multi-step origination, and pluggable zero-knowledge programs that define market behavior. Three products sit on top. **Borrow** provides fixed-rate, fixed-term loans with cross-chain collateral. **Prime Trade** extends leveraged credit lines for perpetual trading on Hyperliquid while preserving native yield. **Earn** gives liquidity providers passive fixed-rate returns under a proof-enforced vault mandate.

2 Products

2.1 Borrow

Borrow lets a user lock collateral on one chain and receive a fixed-rate, fixed-term loan on another. Rate, fee, and total repayment are quoted upfront and locked for the life of the loan. Collateral stays on its home chain and continues accruing native yield—no bridging or unstaking required.

2.2 Prime Trade

Prime Trade is DALOC's prime brokerage product: credit origination for holders of yield-bearing assets on Ethereum, Arbitrum, or Base who want perpetual exposure on Hyperliquid [9] without giving up the underlying.

The conventional path requires unstaking, bridging, and depositing—giving up yield and taking on bridging risk. Prime Trade keeps the collateral where it is: the borrower pledges it and the platform opens a fixed-rate credit line routed directly to the trading venue. The borrower never holds the borrowed capital; protocol-controlled margin agents restrict funds to perpetual trading only—withdrawals, spot transfers, and external lending are blocked. DALOC takes no directional exposure; it earns from lending spread and builder fees.

2.3 Earn

Earn gives liquidity providers passive fixed-rate yield. Capital deposited into the Harvest Vault is deployed into loans by an operator whose strategy is bounded by a publicly auditable mandate, verified per-epoch by zero-knowledge proof (Section 8).

3 Protocol Architecture

All products share a single platform layer, extensible to new products without modifying core contracts.

3.1 Order Book

All loan activity flows through a single order book. Borrower requests and solver quotes are structurally identical—both specify collateral, principal, rate, term, and execution path—and the matching engine treats them symmetrically. The matcher selects the best rate for the borrower among criteria-compatible quotes; the settlement proof verifies that no better match was skipped.

Participation requires no capital commitment. A borrower or solver signs an off-chain intent and grants a token allowance to the hub, authorizing a future transfer but moving nothing. Capital is transferred only when an epoch's settlement proof executes the match on-chain; until then, funds remain in the participant's wallet.

3.2 Criteria-Based Matching

Beyond price compatibility, a match must satisfy *criteria*: typed attributes attached to an intent — counterparty allowlists, jurisdictional restrictions, KYC-tier requirements, or sanctions-screen attestations. The matcher treats a pair as compatible only if each side's criteria are satisfied by the other's attributes.

There is no separate permissioned market; entries in one order book vary only in how restrictive they are.

Criteria originate from three sources: the intent author (self-asserted), on-chain attestations or signed claims (counterparty attributes), and the market’s program hashes (market-level). Criteria can be committed-only — stored as a hash, evaluated inside the proof, revealing only the boolean outcome. The default is open: unrestricted intents match against anyone.

3.3 Markets

DALOC is organized around isolated, immutable lending markets. Each market is identified by a deterministic hash of seven parameters:

$$\text{marketId} = \text{keccak256}(c, \text{chainId}_p, p, h_{\text{rate}}, h_{\text{liq}}, h_{\text{di}}, h_{\text{cmi}}) \quad (1)$$

where c is the collateral asset, chainId_p the principal chain, p the principal asset, and $h_{\text{rate}}, h_{\text{liq}}, h_{\text{di}}, h_{\text{cmi}}$ are zero-knowledge program hashes governing interest accrual, liquidation execution, debt-issuer state transition, and collateral-manager-issuer state transition respectively. All parameters are fixed at creation: a different rate model, liquidation rule, collateral policy, or criteria requirement constitutes a different market.

Markets are created permissionlessly through the *hub* contract (DalocHub). The hub is the sole token rail: all user-facing ERC-20 transfers route through it, creating a single authorization point for capital flows, intent cancellations, and global pause.

Different markets can encode different collateral strategies: a *classical* market locks collateral as a liquidation hedge, while a more aggressive market can permit managed rehypothecation. The choice is a market-definition decision, not a protocol-level toggle.

Each market deploys issuer contracts on both the collateral and principal chains; each funded loan creates per-loan contracts at deterministic CREATE2 addresses. Per-loan contracts are isolated—one per loan, not shared across borrowers—so one position’s risk cannot cascade to another. Section 4 details the contract hierarchy, settlement programs, and epoch mechanics.

3.4 Hook Composition

Lending operations are expressed as atomic hook chains attached to settlement events. Each transfer in the settlement tree carries optional *pre-hooks* and *post-hooks*: pre-hooks execute before the transfer (e.g. deploy a CDP), the transfer moves collateral or principal, and post-hooks execute after (e.g. supply to an external venue, borrow, bridge). All steps must succeed or the entire operation reverts.

Each hook is assembled by the solver and committed into the settlement tree before on-chain execution. A hook's permitted operations are constrained by the market's ZK programs: the settlement proof verifies that every hook is authorized and that the resulting state satisfies the market's invariants. Borrowers sign high-level intents; solvers compete to attach the cheapest valid execution plan. Intent cancellation is always available (even when the protocol is paused) and takes effect at the next epoch boundary.

3.5 Solvers

The Harvest Vault operator is the canonical solver, participating in the order book like any other.

Independent solvers coexist on the same book. A *risk-taking* solver funds from its own balance sheet; a *passive* solver routes through existing money-market liquidity (Aave [2], Morpho [6], Spark [12]) by deploying pledged collateral on established venues. Both modes produce structurally identical entries; the best terms win. Routing and capital sourcing are solver-level concerns, making the platform extensible to new venues without core contract changes.

4 Settlement and Verification

4.1 Epoch Mechanics

Time advances in *epochs*: bounded settlement windows in which the off-chain engine matches intents, runs state-transition programs, and commits a snapshot on-chain. Epoch duration is configurable per market, trading latency against proof cost.

Each snapshot carries five Merkle roots: executable transfers, internal market state, debt records, liquidation events, and CDP deployments.¹ Posting a new epoch requires a valid *fact* in the facts registry, attesting correct state transition via zero-knowledge proof.

Execution is inclusion-proof driven: anyone can execute a committed leaf by supplying a Merkle proof against the epoch root stored on-chain.

Each epoch advance processes three concurrent generations: finalizing loans from two epochs prior, settling principal from one epoch prior, and matching new intents. When the solver is the vault operator, all three compress into a single epoch via an atomic multicall.

¹The internal market state uses a Poseidon-based [7] Merkle Patricia Trie for ZK-friendly state management; the other four roots use keccak-hashed binary trees for on-chain proof verification.

4.2 Settlement Programs

On-chain contracts are settlement executors: they verify Merkle proofs and execute committed transfers and hook chains but contain no market-specific logic. All behavior — interest computation, liquidation detection, oracle selection, collateral policy — lives in zero-knowledge programs whose outputs the contracts verify.

Programs split by economic cadence: **Matching** is one-time per loan (transfers, debt issuance, CDP authorization). **Interest accrual** is periodic (per debt per time window). **Liquidation** is event-triggered (health breaches produce liquidation leaves). **Close** is terminal after repayments and accrual catch up. **Compliance** is periodic per vault epoch (Section 8).

All programs submit results through a single *facts registry* that accepts a fact hash and returns whether it is valid. Two fact types carry all market state:

A **state-transition fact** authorizes an epoch advance:

$$f_{st} = \text{keccak256}(h_{prog}, chainId, issuer, epoch_{prev}, epoch_{next}) \quad (2)$$

where $epoch_{prev}$ and $epoch_{next}$ each encode the full quintuple of Merkle roots. An **interest fact** authorizes accrual on one debt:

$$f_{int} = \text{keccak256}(h_{rate}, chainId, debtId, t_{from}, t_{to}, \Delta I) \quad (3)$$

where ΔI is the interest amount over the accrual window $[t_{from}, t_{to})$. The on-chain contract reconstructs the expected hash from its immutable state, checks the registry, and applies the values only if valid.

4.3 Market and Loan Contracts

Each market has two contracts. The **collateral manager issuer** (collateral chain) advances epoch state, executes Merkle-authorized transfers, and deploys per-loan collateral managers. The **debt issuer** (principal chain) advances epoch state and deploys per-loan issued debt tokens. Neither contains market-specific logic.

Each loan also has two contracts. The **collateral manager** (CDP) holds pledged assets and executes hook-composed pipelines; permitted operations are defined by the market's ZK programs. The **issued debt** is a per-loan ERC-20 tracking accrual via verified facts and partitioning repayment into principal reserve, interest reserve, and protocol fees (Section 7).

4.4 Oracle Freedom and Permissionless Markets

Oracle logic is embedded in the market's zero-knowledge programs. Programs read price or rate data from on-chain contracts via storage proofs, enforcing

freshness inside the proof rather than trusting off-chain feeds. Markets can be created permissionlessly: the protocol verifies program outputs but does not audit program logic. Assessing a market’s risk properties is the participant’s responsibility.

4.5 System Invariants

The architecture maintains three invariants across all states:

1. **Token conservation.** For every asset, total balances across all market contracts equal deposited collateral plus supplied principal minus withdrawals. No operation creates or destroys tokens.
2. **Epoch monotonicity.** Each market’s epoch advances strictly by one, only when the facts registry contains a valid state-transition fact linking the previous and next roots (Equation 2). No epoch can be skipped, replayed, or forked.
3. **Position isolation.** Each loan’s contracts are independently deployed, independently liquidatable, and share no mutable state with any other loan. One position’s insolvency cannot affect another.

5 Loan Lifecycle

5.1 Origination

A borrower posts a request, solvers respond with quotes, and the accepted quote is committed into an epoch’s settlement tree and executed atomically. Four origination pipelines exist, each assembled as an atomic hook chain:

Vault-funded. The vault transfers USDC principal directly to the borrower (or to the margin agent for Prime Trade positions). The collateral manager holds collateral without supplying it to an external venue; no hedging buffers are reserved.

Externally-sourced. The hook chain deposits collateral into the collateral manager, supplies it to the external venue, borrows USDC against it, and transfers the USDC to the borrower. A rate buffer is escrowed to hedge the mismatch between the borrower’s fixed rate and the venue’s variable rate.

Externally-sourced with collateral swap. For long-tail collateral not listed on the external venue (e.g. LDO, UNI): a swap hook converts the collateral to a venue-listed intermediate (e.g. LDO → WETH), then the chain proceeds as above. Both a rate buffer and an option buffer are reserved; the option buffer covers the round-trip swap cost at close.

Hyperliquid margin. For Prime Trade positions, the hook chain follows either pipeline above but diverts at the final step: instead of transferring USDC to the borrower, a bridge hook burns it via CCTP [3] to a per-loan margin agent on HyperEVM, which deposits it as margin (Section 10). Collateral never leaves

Ethereum.

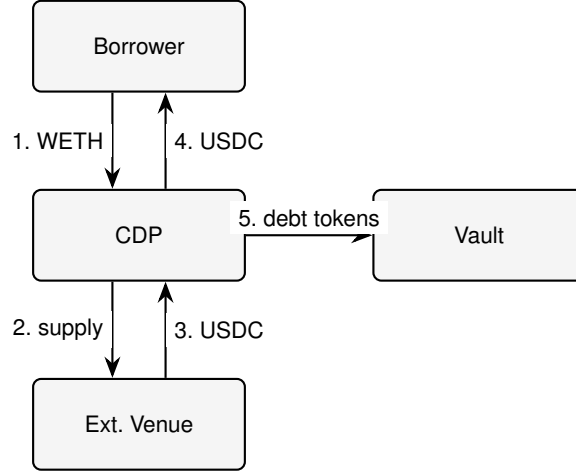


Figure 1: Origination, externally-sourced pipeline. Steps 1–4 execute as a single atomic hook chain. Debt tokens (step 5) are minted to the vault; the rate buffer is escrowed. Vault-funded origination omits the external venue and transfers principal directly.

5.2 Refinancing

A borrower refinances by posting a single refinancing intent. The matching engine treats it identically to an origination quote. On match, the existing loan closes and a new one originates atomically within the same epoch. When the Harvest Vault is creditor on both sides, refinancing is a pure accounting event—no capital moves, only terms change. When the creditor differs, repayment and new funding settle within the same settlement tree.

5.3 Close and Liquidation

On voluntary close, the borrower deposits USDC into the loan’s debt contract. After interest accrual is verified (Section 7), the vault redeems its position and a close hook chain atomically repays the variable debt on the external venue, withdraws the collateral, and returns it to the borrower (with a reverse swap for collateral-swap loans). Any hedging surplus stays in the vault as LP yield. Vault-funded loans skip the external unwind: the borrower repays, and collateral is released directly.

Liquidation triggers when the health factor drops below 1:

$$HF = \frac{C_{value} \cdot \theta}{D} < 1 \quad (4)$$

where C_{value} is the collateral’s current market value, θ the loan’s liquidation threshold (set at origination from the lender’s maximum LTV), and D the outstanding debt.

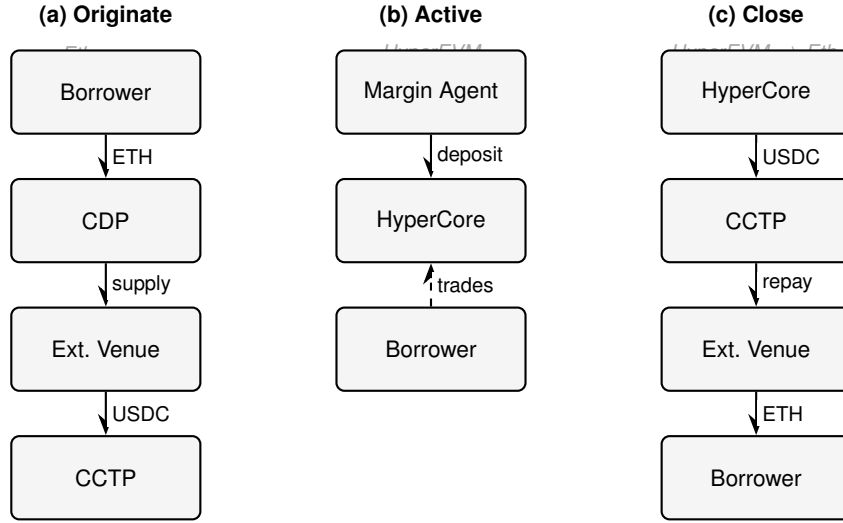


Figure 2: Prime Trade lifecycle. **(a)** Borrower pledges ETH; CDP supplies to the external venue; USDC bridged via CCTP. **(b)** Margin agent deposits into HyperCore; borrower trades (dashed). **(c)** Capital recalled via CCTP; venue repaid; collateral released.

Liquidation spans two epochs. In the first, a proof attesting $HF < 1$ is committed as a leaf in the epoch’s liquidation root—detection is identity-agnostic, requiring only a proof that barriers have been breached.

In the second, the liquidator identifies itself on-chain, repays outstanding debt R via the debt token contract, and seizes collateral worth $R \cdot (1 + \beta)$ from the CDP, where β is the liquidation bonus fixed at origination. Partial liquidation repays the minimum R that restores the health factor above 1; full liquidation repays the entire debt and seizes all remaining collateral. Separating detection from seizure ensures collateral moves only after debt repayment is recorded on-chain.

Because each loan is isolated, bad debt cannot cascade across positions. Any shortfall reduces vault NAV through impaired debt-token value (Section 8).

6 Capital Management

The origination pipelines (Section 5) differ fundamentally in how much capital the vault must commit.

6.1 Vault-Funded Path

The vault commits principal at 1:1. No variable-rate leg, no hedging buffers. Yield per loan is higher but capital leverage is lower.

6.2 Externally-Sourced Path

The vault commits only 3–7% of principal as hedging reserves (rate buffer plus option buffer), achieving 15–30× capital leverage. Pledged collateral simultaneously secures the DALOC loan and the external venue position. External venues are suppliers, not competitors: the platform aggregates their liquidity rather than competing for deposits.

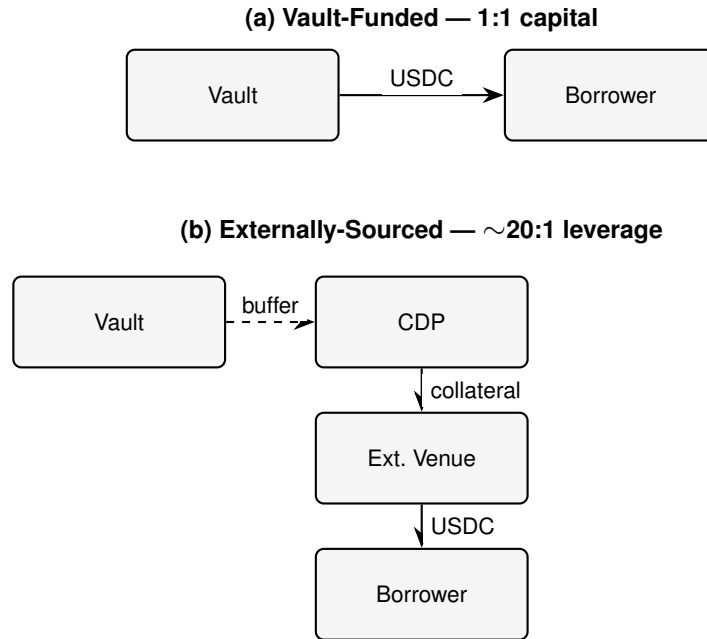


Figure 3: Capital paths. **(a)** Vault-funded: the vault transfers principal directly; no external venue, no hedging buffer. **(b)** Externally-sourced: the CDP borrows from the external venue; the vault commits only hedging capital (dashed).

6.3 Capital Efficiency

The risk engine sizes both buffers at quote time: the rate buffer from tail simulations of variable-rate paths, the option buffer from simulated round-trip swap execution against on-chain pool depth.

Consider two scenarios for a \$500,000 loan backed by WETH collateral at 70% LTV:

Vault-funded. The vault transfers \$350,000 USDC directly to the borrower. Capital committed: \$350,000 at 1:1. The vault earns the full 8% fixed rate on that principal, approximately \$6,900 over a 90-day term.

Externally-sourced. The external venue supplies the \$350,000 principal. The vault reserves \$17,500 as a rate buffer (5% of principal), approximately **20:1 capital leverage**. The vault earns the buffer surplus: the difference between the reserved \$17,500 and the realized variable-rate cost, typically \$3,000–\$8,000 over a 90-day term.

At close, buffer surplus remains in the vault as LP yield; buffer shortfall reduces vault NAV.

7 Position Tokens and Secondary Markets

Each funded loan produces a tokenized position representing the lender's claim. These tokens carry accrual, enforce solvency rules, and trade on secondary markets.

7.1 Debt Tokens

Each funded loan issues a transferable ERC-20 whose face value is one unit of the principal asset per token. The token is the lender's economic position. The claim value per token v (the amount redeemable at maturity) is:

$$v = 1 + I_{\text{cum}} \quad (5)$$

where I_{cum} is the cumulative interest accrued per token, rising as interest is verified. Interest accrues linearly at the loan's fixed rate:

$$\Delta I = D \cdot r \cdot \frac{\Delta t}{T_{\text{year}}} \quad (6)$$

where D is the outstanding debt in token units, r the annualized rate, Δt the accrual period in seconds, and $T_{\text{year}} = 365.25 \times 86,400$. Each accrual requires a valid interest fact (Equation 3) attesting the computed amount.

Repayment proceeds enter a principal reserve. A subsequent accrual step, backed by a proof tied to the market's rate program, splits the interest portion into a protocol fee and an interest reserve. Holders redeem by burning tokens and receiving principal plus their share of the interest index.

The contract maintains a strict accounting identity:

$$R_{\text{principal}} + R_{\text{interest}} + F_{\text{protocol}} = B_{\text{token}} \quad (7)$$

where $R_{\text{principal}}$ is the redeemable principal reserve, R_{interest} the interest reserve, F_{protocol} the accrued protocol fees, and B_{token} the contract's token balance. Two rules protect solvency. First, redemptions are blocked until interest for the latest period is accrued, preventing exits at a stale index. Second, a loan can be marked closed only when accrual is current and the principal reserve covers all outstanding tokens.

Debt tokens from criteria-restricted markets carry transfer-time criteria (e.g. "KYC tier ≥ 2 ") enforced at the hub-routed transfer step, ensuring restrictions persist after origination. When the Harvest Vault is creditor, vault shares (Section 8) reflect the aggregate claim value across all held debt tokens.

7.2 Secondary Markets

A lender who needs liquidity before maturity sells on secondary markets. Debt tokens are standard ERC-20, tradeable on any venue without protocol modification.

Credit-risk price discovery. Two loans with identical face value can trade at different prices reflecting divergent default expectations — the on-chain analog of a credit spread. Pool-based lending cannot produce this signal because every position earns the same rate. Each loan’s distinct token [11] enables granular risk transfer.

8 The Harvest Vault

The Harvest Vault gives passive LPs pool-style UX while an operator deploys capital into loans under a publicly auditable mandate verified per-epoch by zero-knowledge proof.

8.1 Operator and Mandate

The operator’s strategy is bounded by a **mandate**, a policy committed on-chain as a hash at vault deployment. The mandate specifies:

- **Allowed collateral set:** asset addresses the operator may accept as loan collateral.
- **Per-collateral LTV ceiling:** maximum loan-to-value ratio the operator may offer on each collateral type.
- **Per-tenor rate floor:** minimum annualized rate the operator must charge, ensuring the vault never underwrites below a specified spread.
- **Portfolio concentration caps:** maximum exposure per borrower, per collateral type, and per market as a fraction of vault NAV.
- **Liquidation reserve requirement:** minimum idle capital maintained at all times for liquidation response and withdrawal settlement.

The mandate constrains what the operator *can* do, not what they *must* do; within its bounds, the operator retains full discretion.

8.2 Compliance Proof

Each epoch, the operator produces a zero-knowledge proof that every action satisfied the mandate. The proof takes prior vault state, the epoch’s actions, and the mandate hash as public inputs, outputting a **compliance fact** registered in the facts registry (Section 4):

$$f_{\text{compliance}} = \text{keccak256}(h_{\text{prog}}, \text{chainId}, \text{vault}, \text{epoch}_{\text{prev}}, \text{epoch}_{\text{next}}, h_{\text{mandate}}) \quad (8)$$

where h_{prog} is the compliance program hash, $vault$ is the vault contract address, and $h_{mandate}$ is the committed mandate hash.

8.3 Fee-Gating

The operator earns a spread fee escrowed during each epoch. A valid compliance proof releases the escrow to the operator; otherwise it flows back into NAV. An invalid or missing proof does not halt loan settlement or affect borrowers; only the operator's cut is at risk.

8.4 NAV and Shares

LPs deposit USDC and receive ERC-20 shares. NAV equals idle USDC plus debt-token claim values plus escrowed hedging buffers plus accrued interest, less USDC ring-fenced for pending withdrawals. Share price is NAV per outstanding supply, with a virtual offset against inflation attacks. Yield comes from fixed-rate interest (vault-funded loans), buffer surplus (externally-sourced loans), and liquidation proceeds. Impairments reduce NAV directly via share-price adjustment.

8.5 Epoch Withdrawal

Instant withdrawal is incompatible with capital in active positions. The vault queues exits in four steps:

1. **Request.** The LP burns vault shares and receives *withdrawal receipt* tokens for the current epoch.
2. **Epoch elapses.** After the epoch duration, anyone can advance the epoch.
3. **Settlement.** A permissionless call snapshots the share price for all pending requests in that epoch and ring-fences the corresponding USDC, removing it from the vault's deployable idle balance.
4. **Redemption.** The LP burns receipt tokens and receives the ring-fenced USDC, net of a flat redemption fee.

Until settlement, the LP remains exposed to pool gains and losses. The epoch delay prevents withdrawing at a stale NAV before a loss lands.

8.6 Risk Decomposition

9 Risk Quantification

The preceding sections defined the risk channels (Table 1); this section describes how the risk engine sizes each loan's parameters before capital is deployed.

Party	Risk borne	Why	Compensation
Borrower	Collateral decline	Seeks leverage	Fixed-rate USDC
Ext. venue	Smart-contract, util.	Money market	Variable rate
Rate buffer	Variable > fixed	Rate transform	Surplus
Option buffer	Swap cost > escrow	Execution risk	Escrow ~95%
Vault LP	Default, exhaustion	Absorbs residual	Coupon or surplus
Operator	Mandate violation	Active mgmt	Fee, gated on proof

Table 1: Risk decomposition. Each risk channel is routed to the participant with the comparative advantage to bear it. The rate buffer and option buffer rows apply only to externally-sourced loans.

9.1 Dynamic Underwriting

Every loan quote must bound the lender’s loss before capital is deployed. The risk engine evaluates each collateral asset dynamically — accepting, rejecting, or adjusting parameters — and continuously re-evaluates existing positions. Inputs include liquidity conditions (depth and slippage at trade size), non-quantitative dimensions (smart contract, issuer, and infrastructure risk), and venue conditions for Prime Trade. Maximum borrow, fixed rate, and liquidation thresholds are *outputs of the model*, not static protocol parameters.

9.2 Methodology

The risk engine follows a three-stage pipeline: data collection, statistical modeling, and simulation.

Data collection. The engine ingests live on-chain state (asset prices, lending-venue utilization rates, liquidity-pool depth, and historical return series) directly from the chains where collateral and principal reside.

Statistical modeling. Collected data feeds models that estimate the current volatility regime, tail-risk characteristics, and correlation structure, capturing asymmetric downside behavior and discontinuous price events.

Simulation. Monte Carlo simulations project collateral prices and variable-rate funding costs over the loan horizon. Simulated paths are evaluated against the loan’s liquidation barriers and mechanics (partial versus full close, bonuses, slippage), producing a distribution of liquidation probability and loss given default.

9.3 Pricing

The published LTV and rate are chosen jointly to balance expected return and incremental tail risk given the existing book.

Higher volatility or longer duration lowers the LTV; adding correlated exposure to a concentrated book raises the rate. Positions that reduce aggregate tail

risk receive more favorable terms, incentivizing diversification. The rate never falls below the risk-free return plus the simulated expected loss per dollar lent. With collateral quantity C , spot price S_0 , LTV ratio λ , and origination fee ϕ , the maximum borrow is:

$$L = C \cdot S_0 \cdot \lambda \cdot (1 - \phi) \quad (9)$$

With annualized rate r and horizon T in years, debt at maturity is $D = L(1 + rT)$. The liquidation barrier — the spot price at which the health factor (Equation 4) breaches 1 — is:

$$S_{liq} = \frac{D}{C \cdot \theta} \quad (10)$$

where θ is the liquidation threshold. The simulated probability that the collateral price path reaches S_{liq} over the loan's horizon feeds directly into the quoted rate.

For externally-sourced loans, the rate buffer is sized separately, covering the tail of simulated variable-rate paths on the external venue. The borrower's obligation remains the linear formula above regardless of sourcing path.

10 Cross-Chain Exposure

Cross-chain operation is the default; same-chain loans are a special case. Two concerns arise at different layers.

Authorization (protocol-level): proving that collateral was pledged on the collateral chain so the principal chain's contracts can act on it. DALOC uses ZK storage proofs via the Herodotus Data Processor (HDP) [8], combining storage proofs with MMR-backed chain history into a single verifiable claim.

Token movement (solver-level): physically moving the borrowed asset to its destination. Solvers choose the bridge path; for USDC, Circle's CCTP [3] is the common choice. Bridge hooks compose movement into the origination hook chain.

10.1 Deterministic Addressing

The hub deploys at the same address on every supported chain. A loan can lock collateral on chain A while the issued debt token lives on chain B, coordinated via shared epoch commitments and deterministic CREATE2 addresses. New destinations require only a bridge adapter whitelisted on the hub.

10.2 Hyperliquid Margin Agent

The Hyperliquid margin pipeline (Section 5) routes borrowed USDC via CCTP to a per-loan *margin agent* on HyperEVM at a CREATE2 address computed at quote time. The margin agent deposits USDC into HyperCore as margin—capital never enters the borrower's wallet. Permitted actions are restricted

to perpetual trading; withdrawals, spot transfers, and external lending are blocked.

On close or liquidation, the margin agent recalls capital from HyperCore and burns it via CCTP back to Ethereum. Collateral never crosses to HyperEVM; only USDC round-trips. Figure 2 illustrates the full originate–active–close cycle.

10.3 Effective Leverage

The externally-sourced path creates a leverage multiplier: a borrower depositing 40 ETH (\$80,000 at \$2,000/ETH) borrows \$50,000 USDC at 62.5% LTV, bridged to a margin agent on Hyperliquid for up to $5\times$ venue leverage, controlling \$250,000 notional — $3.1\times$ effective leverage without selling the underlying. The lender remains fully collateralized on Ethereum throughout.

11 Governance and Trust Assumptions

The protocol’s administered surface is deliberately narrow. Markets are immutable, vault contracts are non-upgradeable, and every epoch advance requires a valid fact in the facts registry (Section 4). The two surfaces that require trust are the hub contract and the liveness of off-chain infrastructure.

11.1 Protocol Administration

The hub contract is upgradeable, governed by a multisig with timelock. Upgradeability permits response to critical vulnerabilities in the token rail, at the cost of trust in the multisig. Because the hub is the single authorization point for all capital flows (Section 3), an upgrade can alter routing and pause behavior across every market simultaneously. Pause, unpause, and bridge adapter whitelisting (Section 10) are also multisig actions; new cross-chain routes are therefore permissioned even though markets themselves are permissionless.

11.2 Liveness Dependencies

The settlement engine’s correctness is guaranteed by zero-knowledge proofs (Section 4), but its liveness is not: if the engine halts, no new epochs advance. Existing positions remain safe — collateral stays locked, debt is unchanged, intent cancellation remains available — but undercollateralized positions go undetected until epochs resume.

If no epoch advances within one hour, the protocol enters liveness-failure mode: funds are released for withdrawal and positions may unwind without further settlement proofs.

The risk engine produces LTV limits, rates, and buffer sizes off-chain; the mandate (Section 8) enforces hard floors — rate floors, LTV ceilings, concentration caps — but does not validate the model between those bounds.

Cross-chain USDC movement depends on Circle’s CCTP attester infrastructure (Section 10); attester liveness and censorship properties are inherited, not controlled by the protocol. Cross-chain authorization correctness is cryptographic — HDP produces Merkle proofs over actual chain state — but proof-generation availability depends on the provider.

12 Discussion

12.1 Risks

External venue dependency. Governance changes at an underlying venue—reduced LTV ceilings, modified rate curves—directly affect externally-sourced positions. *Mitigation:* vault-funded path requires no external venue; mandate concentration caps limit per-venue exposure.

Correlated rate and price risk. In a crash, collateral value drops while venue utilization spikes, raising the variable rate precisely when the buffer shrinks. *Mitigation:* rate buffers are sized from tail simulations under the joint distribution (Section 9).

Rehypothecation risk. Markets that permit rehypothecation expose positions to correlated venue failures. The protocol enforces correct execution of the rules, not their prudence.

Smart contract and circuit risk. A hub bug could affect all capital flows; a soundness flaw in a settlement program could permit invalid state transitions. *Mitigation:* independent audits, formal verification of accounting invariants, and on-chain invariant monitoring (Section 4).

Oracle risk. Oracle logic is embedded in each market’s ZK programs. A manipulable oracle can produce incorrect liquidation or accrual facts. *Mitigation:* freshness is verified inside the proof; participants choose which markets to interact with.

12.2 Future Capabilities

Operator ecosystem. Multiple independent operators each run their own vault with a distinct mandate. Competition for LP capital creates pressure toward better risk-adjusted returns; the mandate makes comparison meaningful because strategies are verified per-epoch against declared bounds.

Under-collateralized credit. Protocol-controlled custody (Section 10), where borrowed capital cannot leave the constrained execution environment, extends naturally to credit lines exceeding collateral value—the credit decision reduces to risk-engine parameterization.

Cross-margining. ZK programs can encode netting rules across a borrower’s active positions, recognizing offsetting exposure for reduced buffer sizing with-

out weakening per-loan isolation (Section [3.3](#)).

13 Conclusion

DALOC unifies cross-chain prime brokerage, fixed-rate lending, and passive LP yield into a single platform enforced by zero-knowledge proofs. The design separates trust-minimized surfaces (market logic, settlement, position isolation) from administered ones (vault operation, risk pricing, solver competition), constraining the latter with cryptographic verification. Capital efficiency, institutional compliance, and cross-chain reach coexist without compromising per-position isolation.

References

- [1] Aave. Aave V3 technical paper. Technical report, Aave, 2022. URL https://github.com/aave/aave-v3-core/blob/master/techpaper/Aave_V3_Technical_Paper.pdf.
- [2] Aave. Aave V4 overview. Technical report, Aave, 2026. URL <https://github.com/aave/aave-v4/blob/main/docs/overview.md>.
- [3] Circle Internet Financial. Cross-chain transfer protocol (CCTP). <https://developers.circle.com/cctp>, 2023.
- [4] DefiLlama. DefiLlama — DeFi dashboard. <https://defillama.com>, 2026.
- [5] Euler Labs. Euler v2 whitepaper. Technical report, Euler Labs, 2024. URL <https://docs.euler.finance/euler-v2/about/whitepaper>.
- [6] Mathis Gontier Delaunay, Paul Frambot, Quentin Garchery, and Matthieu Lesbre. Morpho blue whitepaper. Technical report, Morpho Labs, 2023. URL <https://github.com/morpho-org/morpho-blue/blob/main/morpho-blue-whitepaper.pdf>.
- [7] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *30th USENIX Security Symposium*, 2021.
- [8] Herodotus. Herodotus data processor: Provable on-chain data access. <https://docs.herodotus.cloud>, 2024.
- [9] Hyperliquid Labs. Hyperliquid: A performant L1 for a decentralized financial system. <https://hyperliquid.xyz>, 2024.
- [10] Robert Leshner and Geoffrey Hayes. Compound: The money market protocol. Technical report, Compound Labs, 2019. URL <https://compound.finance/documents/Compound.Whitepaper.pdf>.
- [11] Bhargav Nagaraja Bhatt, Paul Frambot, Quentin Garchery, Mathis Gontier Delaunay, Adrien Husson, Paul-Adrien Nicole, Adrien Laversanne-Finot, and Matthieu Lesbre. Morpho midnight whitepaper. Technical report, Morpho Labs, 2025. URL <https://github.com/morpho-org/morpho-midnight/blob/main/morpho-midnight-whitepaper.pdf>.
- [12] Sky (formerly MakerDAO). SparkLend. Technical report, Sky, 2023. URL <https://docs.spark.fi>.